The Spell Duet

EDN owns the session. Spacetime owns the surface.

MVP spec — replacing the beam-lisp Clay view layer with a live Spacetime environment driven through `spacetime live`, a sidecar-grade evolution of `spacetime mcp`.

ora · 2026-08-11 · draft for settlement

EDN / beam-lisp

views · signals · intents
templates · basic macros
mission ontology · state
the **what**

Spacetime

layout · design language
animation · styling
shell-local signals
the **how it looks and moves**

0 Contents

1 Purpose: the duet

The Spell interface is re-founded as a **duet** between two homoiconic languages that each keep what they are best at:

- **EDN (beam-lisp, `.bl`)** owns the session ontology: missions, follow-ups, turns, agent outputs, forms, wiring of intents, durable state, and the templates and basic macros that produce them. EDN terms are data; the interface is a value held, transformed, and submitted by the BEAM side.
- **Spacetime (`.st`)** owns presentation: layout, the design language, styling, animation, transitions, and shell-local reactivity. It is the compiled, live, browser-executed surface the human actually touches.

The seam between them is narrow and typed: an EDN view term compiles to a *mounted Spacetime function instance* whose inputs are EDN→JSON payloads, and any Spacetime signal the EDN references by name is validated at live-environment load time. If the duet disagrees with itself, it refuses to start — it never limps.

1.1 Why Clay goes away

The current view layer (`spell/src/spell/ui.bl` + the `spell.clay` contract stub) renders pure EDN trees to a dumb external renderer over a line-oriented Port protocol:

```edn
{:cmd :frame :tree <tree>}
{:cmd :screenshot :path "..."}
{:cmd :inject :event <ev>}
{:cmd :quit}
```

This was a sound study in boundaries, but as the Spell interface it loses on every axis that now matters:

axis	Clay path	Spacetime path
design language	none — raw props (`:bg`, `:padding`) on EDN maps	first-class: selectors, tokens, transitions, `@flip`, keyframes
reactivity	thunks + refs re-realized per frame by the harness	compiled signal graph; `@data signal/stream`, `@handle`, optimistic arms
interaction	events arrive as raw maps; no kit, no forms	shipped interaction kit: picker / confirm / form / review with async buttons
agent driveability	bespoke protocol, unimplemented client	`spacetime mcp` tool surface (`st_fn_put`, `st_mount`, `st_await`, `st_elicit`)
server state	would have to be invented	LiveView bridge: assigns, typed pushes, "diff data, never templates"
liveness	renderer process, one frame at a time	persistent live environment; hot-swap; self-describing functions

DECISION
Clay is not replaced by a better renderer. It is replaced by a better *relationship*: the EDN side stops describing pixels and starts describing the session; Spacetime, which already knows how to be a live interface for agents, becomes the surface. `spell.clay` and `spell.ui`'s pixel vocabulary are retired; `spell.store`, `spell.fence`, and the BOUNDARY/SELF/AGENT contracts survive unchanged.

1.2 The one-sentence spec

INVARIANT

> A mission is an EDN value. Its rendering is a Spacetime shell. The shell is kept alive by `spacetime live`, a sidecar-grade environment that outlives any single agent client, and every cross-reference between the two languages is proven consistent before the first frame is shown.

1.3 Non-goals (MVP)

- No replacement of the Rust compiler or of beam-lisp's evaluator.
- No new reactive system on the EDN side — EDN signals are *references into* the Spacetime signal graph plus Store-backed durable values, not a third runtime.
- No chat UI. The interface is a mission surface, not a message list.
- No visual REPL / inspector panes (that is `docs/live-repl.md`'s job, later).

2 Context: what exists today

This spec assumes the Spacetime improvement plans already recorded in `verse/!tasks` are implemented. §3 lists exactly which ones the duet leans on. This section states the ground truth both sides start from.

2.1 The beam-lisp spell

`spell/` is a first-principles recreation of `spell.jank`: a stateful, process-heavy, self-modifying beam-lisp application, organised in clusters:

namespace	status	role
`spell.ui`	implemented	pure EDN view constructors (`el`, `text`), `realize`; retired by this spec
`spell.clay`	stub	renderer Port client; retired by this spec
`spell.store`	implemented	named Agents, reload-proof state, wire taps; kept
`spell.fence`	implemented	unlinked monitored eval with deadlines; kept
`spell.self`	stub	compile/load/verify/revert self-rewrite; kept, extended (§8)
`spell.providers`	stub	model/tool/agent contracts; orthogonal

Relevant primitives that survive:

```beam-lisp
(fence f args timeout-ms)            ;; isolated, deadline-bound eval
(state-ensure :session [])           ;; named, code-independent state
(state-swap! :session (fn [s] ...))
(tap! :frame-log value)              ;; bounded wire-tap ring buffer
(tap-contains? :frame-log "…")       ;; test discipline: assert on the wire
```

> NOTE
> The wire-tap discipline — assert on emitted frames/commands, not app internals — generalises cleanly: in the duet, the "wire" is the set of MCP tool calls and signal envelopes sent to `spacetime live`. `spell.store/tap!` stays the testing backbone.

2.2 Spacetime: the live environment (`spacetime mcp`)

`spacetime mcp` is a stateful, line-delimited JSON-RPC 2.0 server over stdio, registered with Spell via `spell.kdl`:

```kdl
mcp {
    server "spacetime" type=stdio {
        command "cargo"
        args "run" "--quiet" "--" "mcp"
    }
```

```
6 }
```

It owns a `LiveStore` (functions, instances, event sequences, cursors) behind a lazily started HTTP live plane. Browser actions enter one generic sink (`POST /__mcp/signal/{id}`); the agent consumes them with bounded pull (`st_await`). The tool surface the duet builds on:

tool	purpose
`st_fn_put`	register/update live `.st` function source + revision + compile artifacts
`st_mount`	mount a function as a browser instance; returns URL / input / events resource
`st_await`	bounded wait for the next agent-audience event from an instance
`st_inspect`	contract, diagnostics, input, event history of functions/instances
`st_unmount` / `st_fn_delete`	lifecycle teardown
`st_env_open` / `st_tab_open` / `st_env_close`	environment + tab management
`st_elicit`	server→client typed prompt (`elicitation/create`)
`st_workbench`	mount the unified workbench shell

The interaction kit (`stdlib/__mcp__/kit/{picker,confirm,form,review}.st`) is already ordinary `.st` functions over this machinery: `@mcp-input` declares the payload, `@mcp-action` marks async buttons, `@mcp-submit` collects form values into `payload.values`, and the agent's `st_await` returns the result. Markdown rendering ships as `stdlib/md` (`render-markdown`, snarkdown-based).

2.3 Spacetime: the LiveView bridge (`elixir/spacetime_lv`)

The Elixir bridge proves the second half of the duet: Spacetime as a *component inside a state-owning host*. The thesis, verbatim from the architecture research: "The host is config. The page is the program."

spacetime
```
@host $counter : live("SpacetimeLvWeb.CounterLive")

@data subscribe $count from $counter : count ;

@data stream $fx from $counter {
  receive to Effect { "flash" => Flash($.payload.message); }
}

@data signal $inc() to $counter {
  send emit "inc"
  receive to IncResult { "ok" => Bumped($.reply as number); _ => Failed($.reply); }
  policy latest
}
```

beam-lisp
```
;; the mirror-image contract on the Elixir side (use-macro DSL):
;;   events do event(:inc) end
;;   assigns do assign(:count, :integer) end
;;   pushes  do push(:flash, message: :string, kind: :atom) end
```

Assigns flow server→browser as diffed `st-set` payloads into compiler-registered setters (`phx-update="ignore"` — templates are never diffed, only data). Events flow browser→server through

`handle_event` with correlated `{:reply, …}` decode. A `<Module>.contract.json` sidecar + SHA-256 fingerprint lets the Rust checker prove the seam. This is the exact pattern the duet generalises: *half the state machine in the BEAM, half in the browser, and the compiler proves the seam.*

2.4 Spacetime signals: the canonical model

The S1 signal model (PLAN-038, FEAT-110) is the shipped direction and the duet's contract vocabulary:

1. `@host` — transport/config binding (`http`, `ws`, `live`).
2. `@data signal` — one correlated reply; `@data stream` — many.
3. one matcher family shared by `@match`, `receive` decode, and `@handle`.
4. `@handle $sig { optimistic … receive … }` with scoped cascade.
5. `policy latest|queue|parallel|drop [timeout] [retry]`.
6. `as Type` ascription; `receive to Sum` names the response sum; `final` marks terminal stream arms.

The compiler registry introspection this enables — a function exposing its inputs, transports, and response variants without a hand-fed contract — is what the EDN load gate (§5.4) queries.

3 Required Spacetime improvements

Two classes. **Assumed** improvements are already planned in `verse/!tasks` and are taken as given (this spec only names the dependency). **New** improvements are what the duet itself requires and does not get for free.

3.1 Assumed (planned upstream)

IMP-A1 Self-describing function interfaces ASSUMED IMPLEMENTED

FEAT-111 / PLAN-038: a mounted function exposes its `@mcp-input` payload shape, signal registry, transports, and response sums through `st_inspect` and MCP resources. The duet's load gate (§5.4) reads this registry; without it there is no load-time consistency proof.

IMP-A2 Contract-stable hot-swap in the live environment ASSUMED IMPLEMENTED

FEAT-124: `st_fn_put` with a compatible contract hot-swaps a mounted function without dropping its instance or event history. The duet needs this so the shell (design language) and the session (EDN state) can evolve on independent clocks.

IMP-A3 Automatic event surfacing ASSUMED IMPLEMENTED

PLAN-040 / FUP-168: `resources/subscribe` produces real `notifications/resources/updated`; instance event streams surface into the Spell session without polling. `st_await` remains the explicit fallback. The duet's "submission arrives the moment the user clicks" experience depends on this.

IMP-A4 Interaction kit with async buttons ASSUMED IMPLEMENTED

PLAN-043 / PLAN-044: picker/confirm/form/review as plain `.st` functions with `@mcp-input`, `@mcp-action`, `@mcp-submit`. The duet's forms and fast-follow async buttons are kit instances, not new machinery.

IMP-A5 LiveView bridge: contract emission + host skeleton ASSUMED IMPLEMENTED

FEAT-160 (server-emitted contracts), FUP-121 (compiler-emitted host DOM skeleton), FEAT-138 (Mix compiler + hot `.st` reload). Required for the BEAM-resident durable tier (§4) to host `.st` pages without hand-mirrored skeletons or manual cargo builds.

3.2 New (this spec)

IMP-N1 `spacetime live` — a sidecar-grade environment verb NEW — THIS SPEC

A new CLI verb beside `mcp` and `workbench`, same process shape as `spacetime mcp` (stdio JSON-RPC + HTTP live plane), with three changes of character:

1. **Named, persistent sessions.** `spacetime live <session>` opens (or re-opens) a named live environment. Environment state — functions, instances, event history, cursors — snapshots to `<session>/.live/*.json` on every mutation and restores on boot. Client disconnect no longer implies state loss; the sidecar keeps the whole thing.
2. **Multi-attach.** More than one MCP client (the Spell agent *and* the beam-lisp session process) may attach to the same session; each gets its own cursor over the shared event history.
3. **Load gate hook.** On session open and on every `st_fn_put` into a gated environment, a registered external checker runs before the function goes live (§5.4). Fail → the function stays parked and the gate's diagnostics are returned as the tool result.

Non-goals: no CDP wave, no workbench changes, no new transport — it *is* the mcp server, with persistence, naming, and a gate.

IMP-N2 Dual-destination action routing

NEW — THIS SPEC

Today an async button's envelope has exactly one audience: the agent that called `st_await`. The duet needs each action to declare its destination at authoring time:

```
@mcp-action $approve : to agent ;        // st_await resolves (today)        spacetime
@mcp-action $retry   : to log ;          // appends to session log stream only
@mcp-action $refine  : to agent+log ;    // both
@mcp-action $run-op  : to intent "spell.ops/refine" ;  // fast-follow (§9)
```

`to log` envelopes are recorded into the session event history and rendered by the shell's log stream, but resolve no waiter. This is a small, backward-compatible envelope extension (`audience` field, default `agent`) plus shell rendering support.

IMP-N3 Markdown slots in mounted shells

NEW — THIS SPEC

`stdlib/md` renders markdown strings today. The shell (§5.2) needs markdown *slots*: named regions whose content is data supplied per instance (mission statement, output bodies), hot-swappable without recompiling the shell. Concretely: `@mcp-input` fields typed `markdown`, rendered through `render-markdown` inside a slot node. This is stdlib work, not compiler work.

IMP-N4 Session log stream component

NEW — THIS SPEC

A stdlib shell component rendering the session event history as an append-only, grouped, collapsible log (tool calls, gate verdicts, `to log` button events), driven by an `@data stream` fed from the live plane's event feed. The deeper-log affordance of the MVP experience (§6) is this component. Mostly stdlib; requires the live plane to expose the per-session event feed as a subscribable stream (a thin read over `LiveStore` history + notifications from IMP-A3).

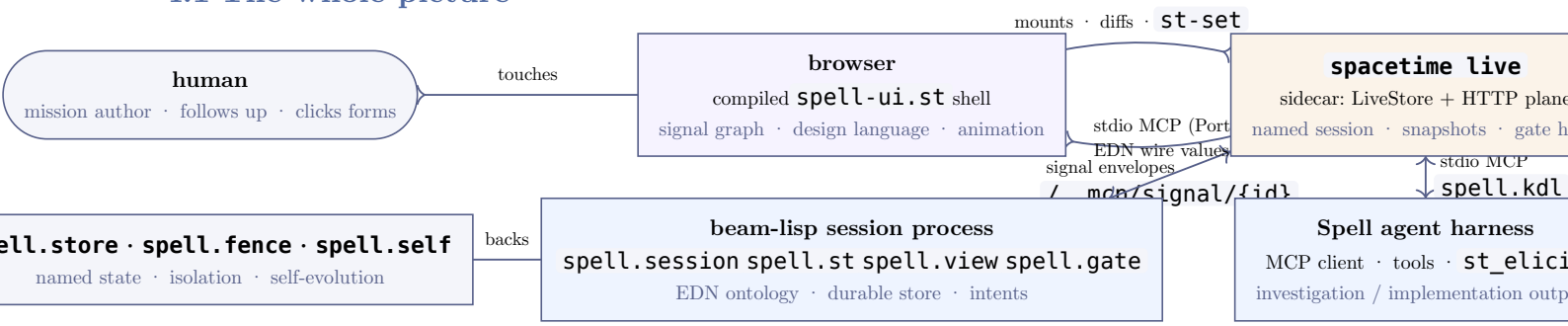
IMP-N5 EDN wire values in tool arguments

NEW — THIS SPEC

beam-lisp's native tongue is EDN, not JSON. `spacetime live` accepts an `edn` media type on `st_fn_put` input payloads and `st_mount` inputs, parsed by the existing bounded EDN reader and normalised to the same internal values. This keeps the duet's wire honest: EDN all the way from the BEAM to the gate, JSON only where the browser already speaks it. (Small; can be fast-followed if it threatens the MVP.)

4 Architecture

4.1 The whole picture



Three processes, one seam. The **sidecar** (`spacetime live`) is the only always-on process: it holds the session environment — mounted functions, instances, event history — across client churn. The beam-lisp session process and the Spell agent harness are both *clients* either may die and reattach without the human's screen so much as flickering.

4.2 Ownership split

	EDN / beam-lisp	Spacetime
owns	session ontology (mission, follow-up, turn, output, log entry) · durable state · intent registry · wiring · view templates · load gate	shell layout · design tokens · typography · animation · transitions · shell-local signals (hover, open/closed, scroll) · kit components
speaks	EDN values; MCP tool calls; intent names	compiled signal graph; JSON envelopes at the browser edge only
fails when	state transitions violate the ontology; an intent is unhandled	a referenced signal or slot does not exist; contract fingerprint drift

> INVARIANT
> One signal graph exists, and it lives in Spacetime. EDN *references* signals (`$turn-open`, `$log-at-tail`, kit action signals) by name; it never implements reactivity. Durable truth lives in `spell.store`; derived liveness lives in the browser; the sidecar holds the join.

4.3 The four seams

1. **Session → sidecar (control).** beam-lisp speaks MCP over a Port's stdio: `st_fn_put` (shell + kit functions, once per boot), `st_mount` (the session shell with the mission as input), tool results via `resources/subscribe` notifications (IMP-A3) or `st_await` fallback. Wire values are EDN (IMP-N5) normalised internally to JSON for the browser.
2. **Agent → sidecar (drive).** The Spell harness attaches as a second MCP client. Its tools are the same `st_*` surface; its outputs arrive as EDN writes to the session ontology (through the beam-lisp process, not directly to the shell) — see §5.3.
3. **Browser → sidecar (signal sink).** Unchanged from today: one generic sink, envelopes routed by destination (IMP-N2) to agent waiters, the log stream, or (fast-follow) named intents.
4. **Sidecar → browser (liveness).** Unchanged: compiled bundles, `st-set` assign diffs, stream pushes. The shell is `phx-update="ignore"` territory; templates are never diffed, only data.

4.4 Why not...

...put the session state in the browser? — The duet's mission state must survive reloads, hot-swaps, and browser crashes, and must be transformable by optics/Specter (§8). Browser signals are the projection, not the truth.

…let the agent write the shell directly? — That recreates chat-with-HTML. The agent writes *outputs into the ontology* the shell decides how ontology renders. Layout authority stays in one place — the `.st` design language — which is what makes the interface feel designed rather than accumulated.
…a Phoenix LiveView in the middle? — The bridge (§2.3) is the proof of pattern, and the fast-follow answer for multi-user or server-rendered deployments. For the single-operator Spell session, the sidecar + MCP already provides everything the bridge provides, with one fewer process and no HEEx anywhere. The EDN gate mirrors the bridge's contract discipline, so adopting the bridge later is a host change, not a redesign.

5 The EDN side: session ontology and views

5.1 The ontology

Everything on screen is a projection of one EDN value, the **session**, held in `spell.store` under `:session`. Its schema (versioned, fingerprinted like a bridge contract):

```edn
{:schema :spell.session/v1
 :mission {:id "m-01"
           :title "…"
           :body {:markdown "…long-form mission statement…"}
           :created-at 1783000000}
 :follow-ups [{:id "f-01" :body {:markdown "…"} :at …}
              {:id "f-02" :body {:markdown "…"} :at …}]
 :turns [{:id "t-01"
          :trigger {:kind :mission :ref "m-01"}     ; or {:kind :follow-up :ref "f-01"}
          :status :done                              ; :running | :done | :failed
          :kind :investigation                       ; or :implementation
          :output {:format :markdown :body "…"}     ; or {:format :forms …} below
          :log [{:at … :kind :tool :summary "…"}     ; deep log entries
                {:at … :kind :gate :verdict :ok :detail "…"}
                {:at … :kind :action :signal "$approve" :payload …}]}]
 :wiring […]                                         ; §5.4
 :intents #{"spell.ops/refine" …}}                   ; fast-follow registry
```

Ontology rules (enforced by `spell.session`, not by convention):

1. A session has exactly one mission; follow-ups only ever append to `:follow-ups`. The attachment timeline is the vector order — nothing else may reorder it.
2. A turn is created by a trigger (mission or follow-up), transitions `:running → :done | :failed` exactly once, and is immutable thereafter. The turn's `:output` is the big end-of-turn artifact.
3. The log is append-only. UI collapse state is a shell signal, never session data.

5.2 The shell: one Spacetime function

The entire interface is one mounted function, `spell/shell`, written in the Spacetime design language. Its input is a projection of the session value; its internals are pure presentation. Sketch:

```spacetime
@fn shell(session Session) {
  @mcp-input session

  @data inline $open-turn string : "" ;      // shell-local: which turn is expanded
  @data inline $log-open  bool   : false ;

  .mission {
    > .title   { text: $.session.mission.title }
    > .body    { markdown: $.session.mission.body }   // IMP-N3 markdown slot
```

```
  }

  .timeline {                                  // follow-ups: the attachment timeline
    @each $f in $.session.follow-ups {
      > .follow-up { markdown: $f.body }
    }
  }

  .turns {
    @each $t in $.session.turns {
      > .turn {
        @on click { $open-turn <- $t.id }
        > .output.hero { markdown: $t.output.body }   // big, end-of-turn artifact
        > .log {
          @when $log-open {
            > stream-log { source: session-events }    // IMP-N4 log stream
          }
        }
      }
    }
  }

  .composer { @mcp-submit $follow-up(fields: [body]) : to agent+log }
}

// ↓ everything below this line is the design language's affair:
//   tokens, type scale, warm-dark palette, entry animations, @flip on
//   timeline append, scroll-spy on the log, … EDN never sees it.
```

The EDN side contains *no layout*. It can name slots (`mission.body`, `turn.output`) and reference signals (`$open-turn`); it cannot express a margin. Conversely the `.st` side contains no session semantics: it renders whatever projection it is fed.

5.3 Turn outputs: markdown and forms

A turn output is either markdown (rendered big, the "hero") or a form collection — the agent's way to ask rather than tell:

```edn
{:format :forms
 :forms [{:id "choose-direction"
          :kind :picker                ; kit function, not new machinery
          :title "Two ways to cut this"
          :options [{:id :a :label "EDN-driven slots" :pros "…" :cons "…"}
                    {:id :b :label "Full .st generation" :pros "…" :cons "…"}]
          :action {:signal "$choose" :to :agent}}
         {:id "confirm-apply"
          :kind :confirm
          :title "Apply the refactor?"
          :action {:signal "$apply" :to :agent}}]}
```

`spell.view` compiles each form entry to a kit mount (`picker.st`, `confirm.st`, `form.st`) with the payload as `@mcp-input`. The kit's async buttons already do the right thing; IMP-N2's `:to` field routes the result.

Because the forms live inside the turn's output region, the user experiences them *in place*, under the output, not in a modal elsewhere.

> **NOTE**
> MVP vs fast-follow, precisely: in the MVP the agent *declares* forms in its output and the beam-lisp process mounts them on the user's next view — submission returns via notification/`st_await` on the agent's next turn boundary. Fast-follow (§9) makes the round-trip intra-turn: the agent emits a form, blocks on `st_await`, and continues in the same turn. The mechanism is identical; only the agent's waiting posture changes.

5.4 The load gate: fail at environment load, not at click time

The duet's central consistency rule. When a session boots (and whenever the shell or wiring is re-`st_fn_put`), `spell.gate` checks, against the sidecar's self-describing registry (IMP-A1):

1. every `$signal` named in EDN (`:action {:signal "$choose" …}`, stream sources, slot bindings) exists in the compiled shell/kit signal registry;
2. every slot named in EDN (`mission.body`, `turn.output`) exists in the shell's `@mcp-input` projection type;
3. every `:to :intent` destination names a registered intent in `:intents`;
4. the shell's contract fingerprint matches the one the session was last gated against — drift is a loud re-gate, not silent skew.

Verdicts are data, appended to the log (`:kind :gate`), and a failing gate leaves the session *parked*: the shell renders the last-good state with a gate banner, and nothing new mounts. This is the EDN-side analogue of the bridge's contract checker (§2.3) and of E0928 handler validation: the seam is proven before it is used.

> **RISK**
> The gate is only as strong as the registry is complete. If IMP-A1′s introspection omits shell-local inline signals, the gate must treat them as a distinct visibility class (EDN may never reference them) rather than silently allowing dangling names.

6 The MVP experience

What it should feel like, as four walkthroughs. Each names the machinery that makes it true, so experience claims stay falsifiable.

1 **Start a session with a long-form mission**

The user opens the session URL (or runs `mix beam_lisp.run spell/src/main.bl -- --session my-mission`, which boots `spacetime live my-mission` and opens the browser). The first thing on screen is not a prompt box — it is the *mission composer*: a full-width, long-form Markdown-ish writing surface. Titles, sections, lists, code fences. Submitting writes `:mission` into the session value, gates, and mounts the shell.

Machinery: composer = kit `form.st` with one markdown field (IMP-A4, IMP-N3); submission routed `:to :agent+log` (IMP-N2) so the act of starting the mission is itself the first log entry.

2 **The interface is a mission, not a chat**

The layout reads top-to-bottom as a document of intent:

1. **Mission hero** — the statement, typeset large, collapsible to a title bar once turns begin.
2. **Attachment timeline** — follow-ups the user adds later appear beneath the mission as a vertical timeline of attachments: small, dated, each one a trigger for a subsequent turn. Writing a follow-up uses the same composer, docked at the timeline's tail.

3. **Turns** — below the timeline, one block per turn. A running turn shows its live status; the deeper log (tool calls, gate verdicts, button events) is one click away per turn, rendered by the log stream component (IMP-N4) — present, inspectable, never shouting.

Machinery: one `spell/shell` function (§5.2); timeline append animated with `@flip`; open/collapse state is shell-local signals, so a page reload re-renders truth from the session value without replaying UI trivia.

3 **The turn ends with a big output — sometimes interactive**

When the agent finishes, the turn flips to `:done` and its output renders *big*: the hero of the turn block, markdown-typeset. When the agent needs the user — a choice between alternatives, a confirmation, a small form — the output is a form collection rendered in place (§5.3): picker cards with pros/cons, a confirm bar, or a field set with a submit button.

In the MVP the agent declares these forms as part of its output EDN; the beam-lisp process mounts them and the user's submission is waiting in the event history when the next turn begins. Fast-follow tightens this to intra-turn (`st_await` mid-turn) and adds async buttons that write straight into the log stream as they are clicked (`:to :log`), so the user can poke at an output — expand a diff, re-run a check, pin an alternative — without involving the agent at all.

Machinery: kit picker/confirm/form (IMP-A4); dual-destination routing (IMP-N2); automatic surfacing of submissions into the Spell session (IMP-A3).

4 **The sidecar keeps the whole thing**

Close the laptop. Kill the agent. Restart the beam-lisp process. Reopen the browser: the mission, timeline, turns, outputs, and log are all there, because they live in the named sidecar session (snapshotted on every mutation, IMP-N1) and in `spell.store`'s durable tier — not in any client. The agent reattaches with its own cursor and sees exactly the submissions it missed.

Machinery: `spacetime live <session>` persistence and multi-attach (IMP-N1); notification cursors (IMP-A3); ontology held as one EDN value (§5.1).

INVARIANT
Experience invariant: at every moment, everything on screen is a pure projection of (session value) + (shell-local UI signals). There is no third source of truth to lose, drift, or contradict the gate.

7 Modules: the settlement to reach

New code is deliberately small. Each module below is one job at one level of abstraction; together they replace `spell.ui` + `spell.clay` (≈200 lines of pixel vocabulary and an unimplemented Port client) with a session vocabulary and a client of an already-shipped protocol.

7.1 beam-lisp side (`spell/src/spell/`)

spell.session ONTOLOGY
Pure functions over the session value plus `spell.store` persistence. Every mutation is a constructor call that validates against the ontology rules; invalid transitions are `{:error …}` data, never exceptions. This is the module the agent's outputs flow through — the agent never writes the shell, it asks `spell.session` to close a turn with an output.
owns: the session schema (§5.1), transition rules, mission/follow-up/turn constructors, output assembly, the `:wiring` table, schema versioning and fingerprints
deliberately not: rendering, transport, signals

spell.st BOUNDARY
The only module that knows MCP exists. Everything else speaks EDN function calls. Reuses the trust-boundary rule already documented for `spell.clay`: incoming lines go through the bounded data reader, never `eval`.
owns: the Port to `spacetime live`, stdio JSON-RPC framing, EDN↔wire encoding (IMP-N5), tool-call wrappers (`fn-put!`, `mount!`, `await-event`, `subscribe!`), reconnect with cursor resume

deliberately not: session semantics, gate logic, what to mount

spell.view TEMPLATE
A pure compiler from ontology to mount payloads. Its output is exactly the `@mcp-input` shape of `spell/shell` and the kit functions — verified against the gate's registry, so "compiles" implies "renders".
owns: session value → shell projection; turn-output EDN → kit mount specs; markdown body handling; the template macros (`defoutput`, `defform`) that keep agent-authored outputs concise
deliberately not: layout, styling, anything `.st`

spell.gate PROOF
Reads the sidecar's self-describing registry (via `spell.st`) and the session's wiring; returns verdicts. Registered with the sidecar as the external gate hook (IMP-N1). The gate is the duet's immune system: it makes inconsistency a load-time event with a name and a log entry.
owns: the §5.4 checks: signal existence, slot existence, intent registration, contract fingerprints; verdict records into the log; the parked/last-good posture
deliberately not: fixing anything

spell.intent INTENT — fast-follow
Async buttons wired directly to beam-lisp operations (§9). An intent is a namespaced, gated, fence-executed function from payload to session mutation. MVP ships the registry shape and the gate check; dispatch turns on in the fast-follow.
owns: the intent registry (`:intents`), dispatch of `:to :intent` envelopes to beam-lisp ops, intent result → session writes
deliberately not: agent policy

7.2 Spacetime side (`spell-ui/` in the session workspace)

spell-ui/shell.st SURFACE
One function, hot-swappable (IMP-A2), contract-stable against the gate's fingerprint. Authored in the full design language; reviewed as design, never edited by the agent.
owns: the whole layout: mission hero, attachment timeline, turn blocks, log drawer, composer dock; design tokens, type scale, palette; all animation
deliberately not: session semantics, durable state

spell-ui/log-stream.st SURFACE
The IMP-N4 component. Fed by an `@data stream` over the sidecar's event feed; stateless beyond scroll/collapse signals.
owns: rendering the session event feed as the deeper log: grouping, collapse, kind badges (tool / gate / action)
deliberately not: event production

7.3 Retired / unchanged

module	fate	why
`spell.ui`	retired	pixel vocabulary with no design language; subsumed by `.st`
`spell.clay`	retired	dumb-renderer protocol; subsumed by `spacetime live` + MCP
`spell.store`	kept	durable named state + wire taps; now backs `:session` and gate tests
`spell.fence`	kept	isolation; wraps intent dispatch and gate checks on untrusted payloads
`spell.self`	kept, extended	self-evolution of `.bl` sources; §8 adds view-term introspection
`spell.providers`	unchanged	model/agent contracts; orthogonal to the view duet

DECISION

> The duet deletes more than it adds: two modules retired, five new ones of which three (`session`, `view`, `gate`) are pure data-and-rules with no I/O. The system's complexity moves from protocol plumbing into one explicitly-checked seam — which is exactly where complexity can be gated, logged, and evolved.

8 Self-evolution through introspection

Part of the infrastructure — not the MVP core — is that the duet can inspect and rewrite itself. The instinct (optics / Specter hooks) is right, and this section states precisely where each tool belongs, because they operate at different levels.

8.1 The three levels of "self"

1. **Session value** — plain EDN data in `spell.store`. Transformed constantly (every turn, every follow-up).
2. **View terms** — the wiring table, form specs, and projections that `spell.view` compiles. EDN data *about* the interface.
3. **Sources** — `.bl` namespace forms and `.st` shell source. Changed rarely, dangerously, and only through `spell.self` (compile → load → fence-verify → revert-on-failure) on the beam-lisp side and gated `st_fn_put` on the Spacetime side.

8.2 Where optics and Specter fit

optics (`priv/optics.bl`) VALUE TRANSFORMS
Level 1 workhorse. Session mutations are optic transforms:
owns: composable paths (`in`, `idx`, `traversed`, `filtered`, `optional`) and focused rewrites (`view`, `over`, `setv`)
deliberately not: selection predicates, short-circuiting

```beam-lisp
;; append a follow-up — one focused rewrite of one path
(optics/over (optics/in :follow-ups)
             (fn [fs] (conj fs follow-up))
             session)

;; close every running turn whose trigger was this follow-up
(optics/over (optics/*> (optics/in :turns) optics/traversed)
             (fn [t] (if (and (= :running (:status t))
                              (= fup-id (get-in t [:trigger :ref])))
                       (assoc t :status :failed)
                       t))
             session)
```

specter (`priv/specter/`) TERM SURGERY
Level 2 workhorse — this is where "register hooks" becomes concrete. A **hook** is a Specter path plus a transform, registered in the session's wiring table and re-checked by the gate:
owns: `ALL`, `keypath*`, `must*`, `pred*`, `srange*`; select / select-first / transform / setval over nested EDN terms
deliberately not: code installation

```beam-lisp
;; hook: every form action routed :to :agent also logs — registered as a
;; wiring transform, applied by spell.view at compile time
(specter/transform
  [:forms specter/ALL :action (specter/pred* #(= :agent (:to %)))]
  (fn [a] (assoc a :also :log))
  output-term)

;; introspection: which signals does this output reference?
```

```
(specter/select
  [:forms specter/ALL :action :signal]
  output-term)   ;; → ("$choose" "$apply") — fed to spell.gate
```

The second example is the important one: Specter is how the gate *finds* the cross-references it must prove. Hooks are data (path + predicate + transform), so they are enumerable, gateable, and revertible — which free-form code hooks would not be.

8.3 What neither library does

Neither optics nor Specter touches Level 3: no source introspection, no namespace-form enumeration, no binary installation. That remains `spell.self`'s contract, blocked on the bounded data reader and compile-string→binary. The duet does not unblock it; it gives it a safer target: evolving the *wiring* and *gate rules* (data, Levels 1–2) covers most of what self-evolution wants day-to-day, and Level 3 stays rare and fenced.

OPEN QUESTION

Should hooks live in the session value (evolvable at runtime, gated) or in `.bl` source (static, reviewed)? This spec puts them in the session value and leans on the gate + log for accountability. If hook behaviour ever needs hot-code properties (closures over evolving code), promote specific hooks to named intents and let `spell.self` own them.

9 Roadmap: MVP → fast-follow → later

MVP **The duet stands up**

1. `spacetime live` verb: named sessions, snapshots, multi-attach, gate hook (IMP-N1), on top of the assumed MCP improvements (IMP-A1…A4).
2. `spell-ui/shell.st`: mission hero, attachment timeline, turn blocks with hero markdown outputs, composer, kit forms mounted per output (IMP-N3 markdown slots; IMP-A4 kit).
3. beam-lisp modules: `spell.session`, `spell.st`, `spell.view`, `spell.gate` — pure ontology + one MCP client + one pure projection compiler + the load gate (§5.4).
4. Experience 1–3 as specced (§6), with form submissions consumed at turn boundaries; `:to :log` routing live (IMP-N2).
5. Test discipline: `spell.store` taps over MCP frames; gate verdicts asserted as log data; one scripted end-to-end session (mission → follow-up → picker → submission) replayed headlessly.

FF-1 **The duet gets conversational**

1. Intra-turn forms: agent emits a form and blocks on `st_await`; async buttons (`:to :log`) write into the log stream on click (IMP-N2 audiences, IMP-A3 notifications).
2. `spell.intent` dispatch: buttons wired directly to beam-lisp ops (`:to :intent "spell.ops/refine"`), fence-executed, results written back to the session; the gate already proves the wiring.
3. Session fork/branch: a mission variant explored in a forked session value, merged by optics transforms — the first real payoff of keeping the interface as a value.

FF-2 **The duet becomes a place**

1. LiveView host option: adopt `spacetime_lv` as the durable tier for multi-operator or server-rendered sessions, using the bridge's contract machinery (IMP-A5) — a host change, not a redesign (§4.3).
2. Self-evolution graduated: hooks promoted to intents, `spell.self` unblocked for Level-3 changes, shell hot-swap driven by gate-approved `.st` proposals from the agent itself (IMP-A2 + IMP-N1 gate hook).

3. Visual REPL / inspector panes for the session, per `docs/live-repl.md`, mounted as regions beside the shell.

9.1 What settles this spec

The decisions marked **DECISION** and **OPEN QUESTION** are the settlement surface:

1. the ownership split (§4.2) — EDN never expresses layout; `.st` never holds session semantics;
2. one shell function vs. many mounted regions (§5.2 chose one function + kit mounts inside output regions);
3. forms as kit instances inside turn outputs vs. a bespoke form system (§5.3 chose kit);
4. hooks as gated data via Specter vs. free code hooks (§8 chose data);
5. hook placement: session value vs. source (§8 open question);
6. MVP form round-trip at turn boundaries vs. intra-turn (§5.3 note, §9).

The interface is a value. The surface is a language.
The sidecar keeps the whole thing, and the gate keeps them honest.